PROYECTO OPENCV

Imagen Digital

JOSÉ ANTONIO RAMOS BARRAGÁN

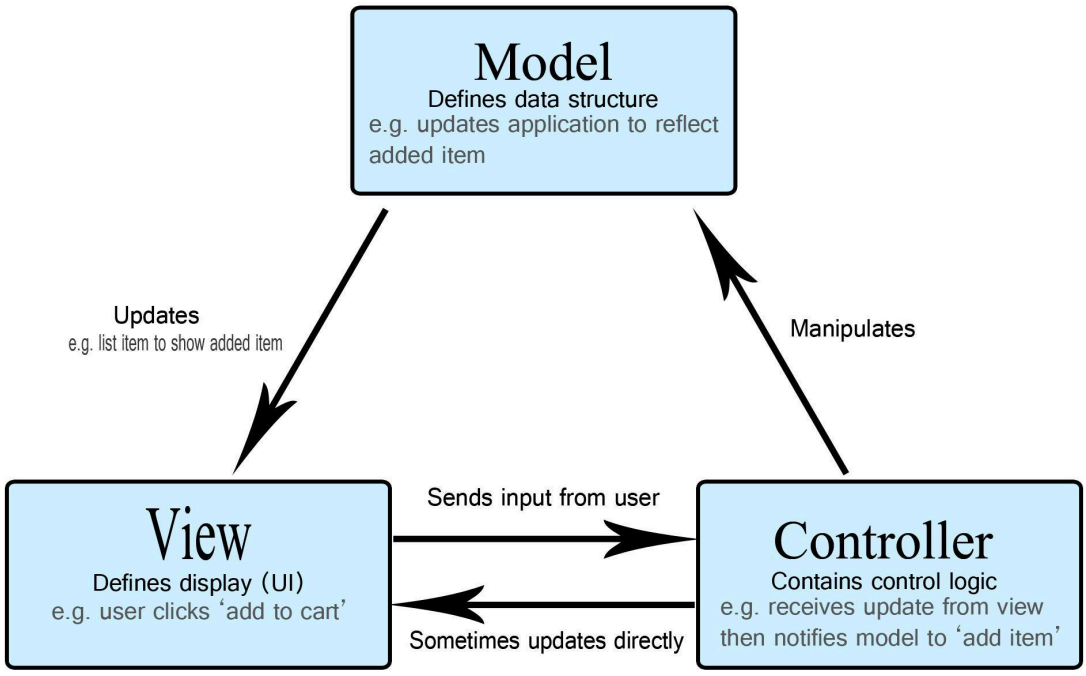
PROYECTO OPENCV **1**
Proyecto de OpenCV **3**
Instalación y ejecución 3
1. Módulos de la implementación **4**
1.1 Captura de la cámara y ciclo de vida del detector (DetectorDeMano.iniciar / _bucle) 4
1.1.1 Detección de la cara (detectar_cara) 5
1.1.2 Rango de color dinámico de la piel (calcular__rango_color_cara) 6
1.1.3 Máscara de piel (calcular_mascara_piel) 7
1.1.4 Exclusión de la cara y el cuello (tapar_cara_en_mascara y pintar_cuadro_cara) 8
1.1.5 Elección del bloque de la mano (elegir_bloque_mano) 8
1.1.6 Separación de la mano y el brazo (separar_mano) 9
1.1.7 Clasificación de mano abierta o cerrada (calcular_solidez) 9
1.1.8 Seguimiento, suavizado y posición normalizada (_procesar) 10
1.2 Ventana de la aplicación y coordenadas virtuales (Ventana) 13
1.3 Bucle principal y máquina de estados (App.ejecutar) 13
1.4 Control del menú con ratón, teclado y gesto de la mano (ControladorMenu) 14
1.5 Control de la nave con la mano (ControladorJuego) 15
1.6 Dibujo del menú (VistaMenu) 16
1.7 Dibujo del juego y composición de sprites con transparencia (VistaJuego) 16
2. Problemas ocurridos y soluciones técnicas **17**
2.1 El espacio de color para detectar la piel 17
2.2 Paredes y sombras confundidas con piel 18
2.3 La cara y el cuello detectados como mano 18
2.4 El brazo desplazaba el centro de la mano 18
2.5 Inestabilidad en el reconocimiento de mano abierta y cerrada 18
2.6 Saltos entre manchas de piel parecidas 19
2.7 Bloqueo de la interfaz y teclas sin estado de pulsación 19
2.8 Tamaño de ventana variable y texto con tildes 19
2.9 Sprites con transparencia 20
3. Mejoras futuras y líneas de trabajo no implementadas **20**
3.1 Calibración inicial del color de piel sin depender de la cara 20
3.2 Robustez frente a fondos con tonos parecidos a la piel 20
3.3 Reconocimiento de más gestos 21
3.4 Predicción del movimiento con filtro de Kalman 21

Proyecto de OpenCV

Juego de naves controlado con la mano mediante visión por computador

El proyecto es un juego clásico de naves en el que muevo la nave y disparo con la mano delante de la webcam. Está hecho con arquitectura MVC, con el modelo, la vista y el controlador separados, y todo el campo de juego se calcula en unidades virtuales de 1280 × 720. OpenCV aparece en tres sitios. El primero es el paquete de visión, en el archivo seguidor_mano.py, que detecta y sigue la mano. El segundo es el controlador, formado por app.py y los controladores de menú y de juego, que se encarga de la ventana, el teclado, el ratón y el bucle principal. El tercero son las vistas, que dibujan cada fotograma. El modelo no usa OpenCV y solo contiene la lógica del juego.

La arquitectura sigue el patrón MVC, tal y como se ve en este esquema.



Instalación y ejecución

Necesito Python 3.10 o superior y una webcam que funcione, ya sea integrada o USB. Lo primero es crear y activar un entorno virtual, después instalar las

dependencias y por último lanzar el juego con main.py. En Windows con PowerShell los comandos son estos:

```
python -m venv id2627virtual
.\id2627virtual\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python main.py
```

En Linux y macOS el entorno se activa con source id2627virtual/bin/activate. Para jugar hay que colocar la mano delante de la cámara, con buena iluminación y el fondo lo más despejado posible, a ser posible blanco. Al mover la mano se desplaza la nave y para salir del juego se pulsa la tecla Esc.

1. Módulos de la implementación

1.1 Captura de la cámara y ciclo de vida del detector (DetectorDeMano.iniciar / _bucle)

Descripción: abre la webcam, carga el clasificador de caras y lanza un hilo propio que va leyendo fotogramas sin parar. El detector solo guarda el último resultado, que incluye la posición de la mano, si está abierta o cerrada, la imagen con lo detectado dibujado y la máscara. Así el hilo principal puede consultarlo cuando quiera sin esperar a la cámara.

Funciones de OpenCV utilizadas y justificación:

- **cv2.CascadeClassifier(NOMBRE_CASCADA):** carga el modelo haarcascade_frontalface_default.xml. Con empty() se comprueba que el archivo existe. Si falta, se guarda el motivo y el juego sigue solo con el teclado, con las teclas WASD.
- **cv2.VideoCapture(indice_camara) e isOpened():** abren la cámara indicada en config.py, que por defecto es la 0, y comprueban que está disponible antes de arrancar el hilo.
- **camara.read():** devuelve el fotograma y un booleano que indica si la lectura ha ido bien. Si falla, se guarda el error y el bucle termina sin romper el programa.
- **cv2.flip(imagen, 1):** voltea la imagen en horizontal para tener efecto espejo, de forma que si muevo la mano a la derecha la nave también va a la derecha.
- **camara.release():** libera la cámara en el bloque finally, incluso si el hilo acaba por una excepción.

Hacia fuera, el detector ofrece cuatro métodos que usa el hilo principal. **obtener()** devuelve la posición de la mano, con x e y entre 0 y 1 o None si no hay mano, y si está abierta o cerrada. **obtener_frame()** devuelve la última imagen de la cámara con lo detectado dibujado y **obtener_mascara()** la última máscara de la mano, y los dos valen None hasta que se procesa la primera imagen. **detener()** pone a falso la bandera de ejecución y espera como mucho 2 segundos a que el hilo termine. El hilo se crea con daemon=True para que nunca impida cerrar el programa.

Antes de adentrarme en las funciones por separado quedo una pequeña imagen con el resumen del proceso:



1.1.1 Detección de la cara (detectar_cara)

Descripción: localiza la cara más grande de la imagen. No sirve para jugar, sino como referencia para dos cosas. Una es sacar el color real de la piel del usuario con la luz del momento y la otra es apartar la cara y el cuello de la máscara para que no se confundan con la mano.

Funciones de OpenCV utilizadas y justificación:

- **cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY):** pasa la imagen a un solo canal de luminancia, que es con lo que trabaja el clasificador de Haar.

- **detector.detectMultiScale(imagen_gris, 1.2, 5, minSize=(80, 80)):** busca caras a varias escalas.
    - El factor de escala 1.2 es un equilibrio entre velocidad y precisión al reducir la imagen en cada pasada.
    - Con minNeighbors igual a 5 hace falta que varias detecciones se solapen para aceptar una cara, y así se evitan los falsos positivos.
    - Con minSize de 80 × 80 se ignoran las zonas pequeñas que no pueden ser la cara del jugador.
    - Si salen varias caras, max() elige la de mayor área.

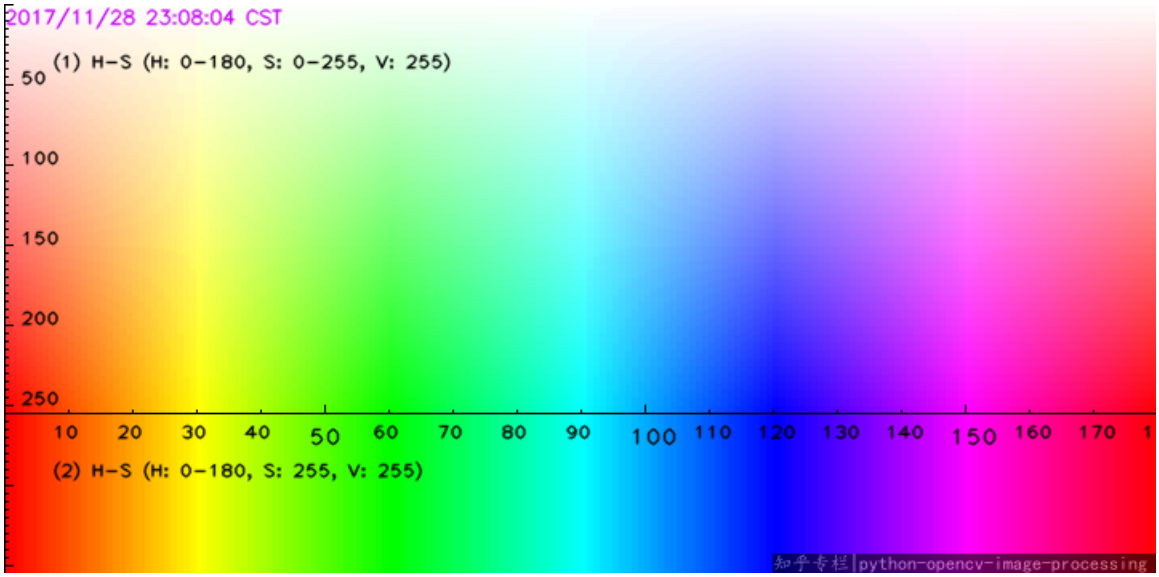
1.1.2 Rango de color dinámico de la piel (calcular__rango_color_cara)

Descripción: analiza una zona de la cara, concretamente la nariz y las mejillas, y calcula el rango de color de piel del usuario con la iluminación actual. Gracias a esto la mano se reconoce aunque la luz no sea la mejor.

Funciones de OpenCV utilizadas y justificación:

- **cv2.rectangle(imagen, (x1, y1), (x2, y2), (0, 255, 0), 2):** en modo depuración dibuja un marco verde en la zona de lectura de color, para comprobar que cae sobre la nariz y las mejillas.
- **cv2.cvtColor(zona, cv2.COLOR_BGR2YCrCb):** pasa la zona a YCrCb, que separa el brillo, el canal Y, del color, los canales Cr y Cb. Esto lo explico con más detalle en el apartado del README sobre por qué uso YCrCb.
    - HSV no me servía porque la piel es rojiza y en HSV el rojo está en los dos extremos de la escala de tono, el 0 y el 179. Eso obliga a calcular la media del tono como un ángulo y a partir el rango en dos con inRange.
    - BGR tampoco, porque el color y el brillo van mezclados y una mano en sombra tiene valores muy distintos a los de la misma mano iluminada.
    - En YCrCb, Cr y Cb son números normales sin vuelta y el brillo queda libre.
- **Nota de indexación directa con NumPy:** se recorta la zona entre el 25 % y el 75 % del ancho de la cara y entre el 30 % y el 75 % del alto, se aplana con reshape(-1, 3) y se calcula la mediana con np.median, que ignora reflejos y sombras puntuales. El rango final deja el brillo libre de 0 a 255 y limita Cr a más o menos 1 y Cb a más o menos 10 alrededor de la mediana.

Esta imagen, que he sacado de omes-va.com, enseña los colores de HSV según el tono en horizontal, de 0 a 179, y la saturación en vertical. Se ve cómo el rojo aparece a la izquierda y a la derecha de la escala.



1.1.3 Máscara de piel (calcular_mascara_piel)

Descripción: genera una imagen binaria en la que la piel es blanca y todo lo demás negro. Para ello combina un rango de color fijo con el rango dinámico de la cara y después la limpia hasta que queda una silueta continua de la mano.

Funciones de OpenCV utilizadas y justificación:

- **cv2.inRange(imagen_ycrcb, PIEL_MIN, PIEL_MAX):** pone en blanco los píxeles que caen dentro del rango fijo de piel, entre 0, 138, 80 y 255, 175, 130. Da una base estable cuando las sombras alteran el color.
- **cv2.inRange(imagen_ycrcb, rango_cara[0], rango_cara[1]) y cv2.bitwise_or:** crean una segunda máscara con el rango dinámico y la unen a la anterior con una operación O. Con esto se rescata la mano cuando hay una luz rara que el rango fijo no cubre.
- **cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5)) y cv2.morphologyEx(mascara, cv2.MORPH_OPEN, KERNEL_ELIPSE, iterations=2):** la apertura elimina los puntos sueltos de ruido. Después se repite con cv2.MORPH_CLOSE, que tapa los agujeros pequeños dentro de la mano. Elijo un kernel elíptico para evitar las esquinas marcadas de uno rectangular.

- **cv2.GaussianBlur(mascara, (5, 5), 0) y cv2.threshold(mascara, 127, 255, cv2.THRESH_BINARY):** suavizan el borde irregular y devuelven la imagen a blanco y negro puro.
- **cv2.findContours(mascara, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE) y cv2.drawContours(mascara, contornos, -1, 255, -1):** dibujan rellenos los contornos exteriores, y así cualquier negro rodeado de blanco pasa a blanco. De este modo se cierran los huecos interiores de la silueta.

1.1.4 Exclusión de la cara y el cuello (tapar_cara_en_mascara y pintar_cuadro_cara)

Descripción: pinta de negro sobre la máscara la zona que ocupan la cara y el cuello. Los dos tienen el mismo color que la mano y competirían con ella como candidatos.

Funciones de OpenCV utilizadas y justificación:

- **cv2.rectangle(mascara_piel, (x, y), (x + ancho, y + int(alto * 1.4)), 0, -1):** dibuja un rectángulo negro relleno, de grosor -1, cuya altura es un 40 % mayor que la de la cara detectada para que cubra también el cuello.
- **cv2.rectangle(imagen, ..., (255, 0, 0), GROSOR_LINEA):** dibuja un marco azul en BGR sobre la imagen de la cámara para ver qué cara se está detectando.
- Si la cara desaparece, se conserva la última detectada durante un único fotograma, según la constante FRAMES_MAX_SIN_CARA, para que un fallo puntual no deje la cara a la vista en la máscara.

1.1.5 Elección del bloque de la mano (elegir_bloque_mano)

Descripción: separa la máscara en manchas independientes, como la mano, un trozo de pared o un objeto de color parecido, y decide cuál es la mano. Da prioridad a la que sigue en la zona donde estaba en el fotograma anterior.

Funciones de OpenCV utilizadas y justificación:

- **cv2.connectedComponents(mascara_piel):** agrupa los píxeles blancos que están pegados entre sí y a cada grupo le asigna un número a partir del 1. El fondo negro es el 0.

- **Nota de indexación directa con NumPy:** con np.bincount se mide el área de cada mancha y se anula la del fondo para que no gane por tamaño. Después se mira la ventana de la mano anterior, ampliada 50 px por cada lado. Si hay piel ahí, se elige la mancha que más aparece y, si no, la más grande de la imagen. Cualquier candidata de 3000 píxeles o menos se descarta como ruido, según la constante AREA_MINIMA_MANO.

1.1.6 Separación de la mano y el brazo (separar_mano)

Descripción: corta el brazo de la silueta buscando el estrechamiento de la muñeca. Así el centro que se calcula después corresponde a la mano y no se desplaza hacia el antebrazo.

Funciones de OpenCV utilizadas y justificación:

- **Nota de diseño e indexación algorítmica:** este módulo no necesita llamadas a la API de OpenCV y trabaja con NumPy sobre la máscara que ya tenemos:
    - Suma los píxeles de piel de cada fila con sum(axis=1) para obtener la anchura por fila.
    - Localiza la punta de la mano, que es la primera fila con piel, y la palma, que es la fila más ancha desde la punta hacia abajo.
    - Recorre las filas desde la palma y marca la muñeca en la primera cuya anchura baja del 90 % de la de la palma. Si el brazo nunca se estrecha, aplica un límite de seguridad de 0.8 veces la anchura máxima.
    - Pone a cero todo lo que queda por debajo de ese límite.

1.1.7 Clasificación de mano abierta o cerrada (calcular_solidez)

Descripción: decide si la mano está abierta o cerrada comparando su silueta con la figura convexa que la envuelve. Una mano abierta deja huecos profundos entre los dedos y baja la solidez, mientras que un puño es compacto y la mantiene alta. Este estado se usa como gesto, con la mano cerrada para disparar y para pulsar botones.

Funciones de OpenCV utilizadas y justificación:

- **cv2.findContours(mascara_mano, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE):** obtiene el contorno exterior de la mano. Me quedo con el de mayor área.
- **cv2.contourArea(contorno) y cv2.convexHull(contorno):** el cociente entre el área de la mano y el área de su envolvente convexa, que es como una

goma elástica que la rodea, da la solidez. El denominador tiene un mínimo de 1 para evitar divisiones por cero.

- **cv2.boundingRect(contorno):** da el ancho y el alto para fijar la profundidad mínima de un hueco válido en 0.1 · max(ancho, alto). Así el valor se adapta al tamaño que tenga la mano en pantalla.
- **cv2.convexHull(contorno, returnPoints=False) y cv2.convexityDefects(contorno, indices_envolvente):** devuelven los índices de la envolvente y, a partir de ellos, los defectos de convexidad, que son los valles entre los dedos. La profundidad llega en punto fijo, así que se divide entre 256, y solo se cuentan los huecos que superan el mínimo.
- **Combinación final:** cada hueco profundo resta 0.08 a la solidez, de modo que con los dedos juntos los valles siguen delatando que la mano está abierta. El valor se suaviza con un filtro exponencial que mantiene un 80 % del histórico y aporta un 20 % del nuevo, y se compara con UMBRAL_SOLIDEZ, que vale 0.6. Por debajo la mano está abierta y por encima, cerrada.

En esta imagen se ve la envolvente convexa de una mano y los defectos de convexidad, que son los huecos entre los dedos. Son la clave para saber si la mano está abierta o cerrada.

[image: Convex Hull y convexity defects]

1.1.8 Seguimiento, suavizado y posición normalizada (_procesar)

Descripción: une todos los pasos anteriores en cada fotograma, mantiene la ventana de búsqueda de la mano, suaviza su caja para que no tiemble, dibuja la información de depuración y devuelve la posición del centro de la mano normalizada entre 0 y 1.

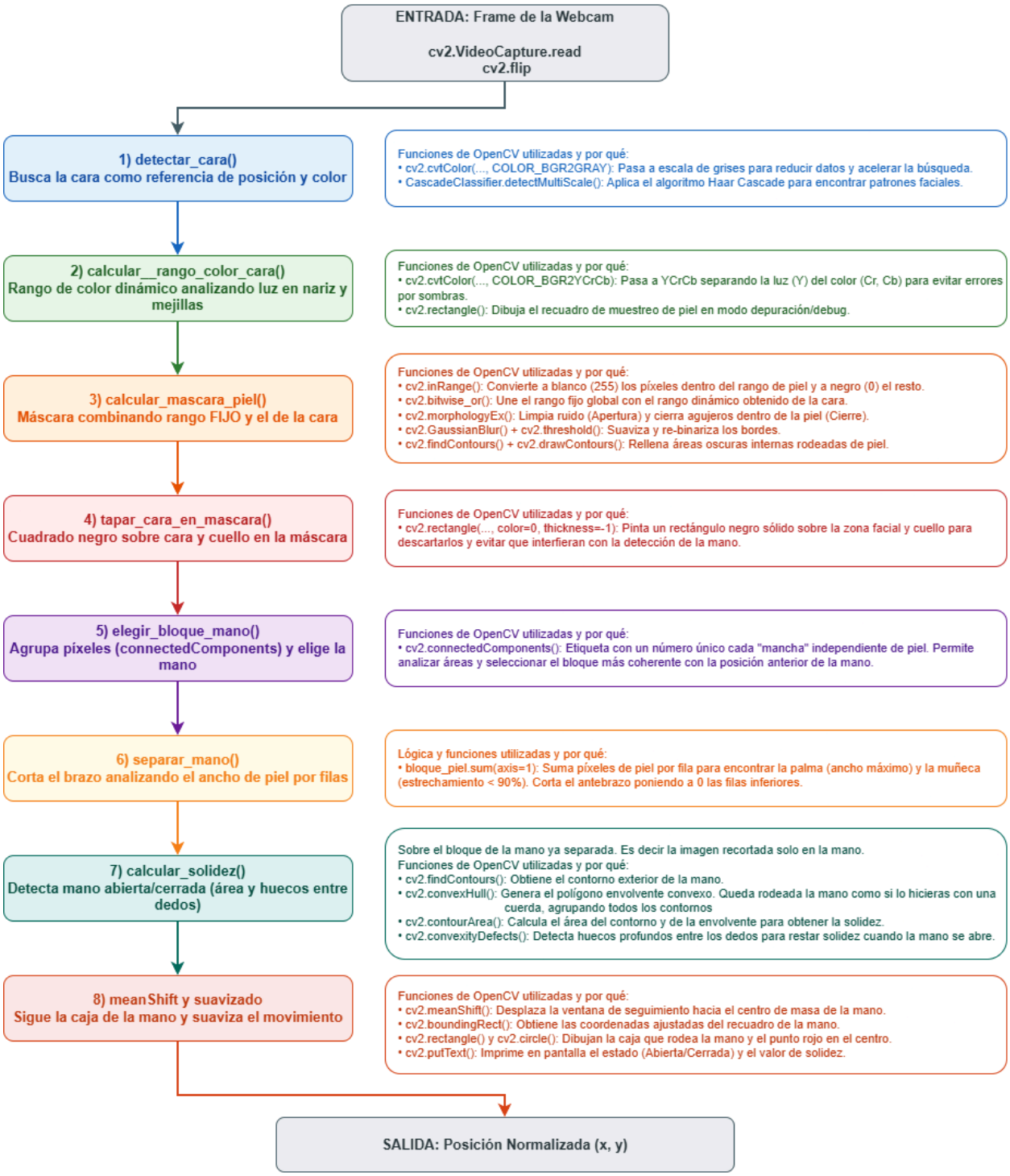
Funciones de OpenCV utilizadas y justificación:

- **cv2.meanShift(mascara_mano, ventana_mano, CRITERIOS_MEANSHIFT):** desplaza la ventana de seguimiento hacia la zona con más píxeles blancos. Los criterios de parada son 15 iteraciones o un desplazamiento menor de 2 px.
- **cv2.boundingRect(mascara_mano):** cuando hay mano calcula la caja envolvente exacta y la guarda como ventana de búsqueda para el siguiente fotograma.

- **suavizar_caja con NumPy:** mezcla la caja anterior al 70 % con la nueva al 30 %, y así el cursor y la nave no parpadean por pequeñas variaciones de la máscara.
- **cv2.rectangle y cv2.circle en pintar_mano:** dibujan la caja amarilla de seguimiento y un punto rojo de 6 px en su centro.
- **cv2.putText(imagen, texto, (20, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2):** escribe sobre la imagen de la cámara el estado, que puede ser Abierta, Cerrada o Sin mano, junto con el valor de solidez.
- **Posición normalizada:** el centro de la caja suavizada se divide entre el ancho y el alto de la imagen para devolver x e y entre 0 y 1, sin depender de la resolución de la cámara.

Todo el flujo de la detección de la mano, con las funciones de OpenCV de cada paso, queda resumido en este esquema.

SEGUIMIENTO DE LA MANO



1.2 Ventana de la aplicación y coordenadas virtuales (Ventana)

Descripción: envuelve la ventana de OpenCV. La crea redimensionable, mantiene la proporción 16:9, muestra los fotogramas, detecta si se ha cerrado y convierte entre píxeles reales y las unidades virtuales de 1280 × 720 en las que se define todo el juego.

Funciones de OpenCV utilizadas y justificación:

- **cv2.namedWindow(nombre, cv2.WINDOW_NORMAL) y cv2.resizeWindow:** crean una ventana que el usuario puede redimensionar y que empieza en 960 × 540.
- **cv2.getWindowImageRect(nombre):** lee el tamaño real del área de dibujo. Si el ancho o el alto valen cero, la ventana está minimizada y no se dibuja nada. Si el alto se aleja más de 2 píxeles de ancho · 720 / 1280, se corrige con resizeWindow para conservar la proporción.
- **cv2.imshow(nombre, imagen):** muestra el fotograma completo que genera la vista activa.
- **cv2.getWindowProperty(nombre, cv2.WND_PROP_VISIBLE):** si devuelve un valor menor que 1, el jugador ha cerrado la ventana y el bucle principal termina.
- **cv2.setMouseCallback(nombre, funcion):** registra la función que recibe los eventos del ratón. Sus coordenadas se convierten a unidades virtuales con pixel_a_virtual.
- **cv2.destroyAllWindows():** cierra todas las ventanas, tanto la del juego como las de la webcam y la máscara, cuando termina el programa.
- Las funciones auxiliares escalar_punto, escalar_rect y escalar_longitud hacen la conversión contraria, de unidades virtuales a píxeles. Gracias a ellas cualquier dibujo se adapta a la ventana sin tocar el modelo.

1.3 Bucle principal y máquina de estados (App.ejecutar)

Descripción: coordina toda la aplicación con un único bucle y una máquina de estados con tres estados, MENU, JUGANDO y PAUSA. Todo lo que tiene que ver con ventanas y teclado de OpenCV ocurre en el hilo principal. La cámara tiene su propio hilo y la actualización del juego a 60 FPS se ejecuta en un tercero.

Funciones de OpenCV utilizadas y justificación:

- **cv2.waitKey(1) & 0xFF:** procesa los eventos de la interfaz de OpenCV y lee la tecla pulsada. La máscara 0xFF deja solo el código de 8 bits y el valor 255 significa que no se ha pulsado nada.
- **cv2.imshow("webcam", frame) y cv2.imshow("mascara", mascara):** abren dos ventanas auxiliares de depuración que muestran la imagen de la cámara con lo detectado y la máscara de la mano. Se activan con la constante MOSTRAR_CAMARA.
- **Nota de diseño concurrente:** _bucle_controlador actualiza el controlador activo y guarda el fotograma resultante en frame_actual. Después espera con time.sleep lo que falte hasta completar el periodo de 1/60 s. El hilo principal solo se limita a mostrarlo, de manera que la lentitud de la cámara nunca bloquea el dibujo.

Control de FPS y tiempo de espera: para que el juego vaya a 60 FPS hay que medir cuánto tarda el trabajo de cada fotograma y esperar solo el tiempo que sobre. Si Δt es lo que ha tardado el fotograma, medido antes de esperar, los FPS reales, el tiempo objetivo y el tiempo restante son:

$$\Delta t_n = t_n - t_{n-1} \qquad \text{FPS} = \frac{1}{\Delta t_n}$$

Como queremos que vaya a 60 FPS:

$$\text{FPS}_{\text{objetivo}} = 60$$

Despejamos Δt_n:

$$T_{\text{objetivo}} = \frac{1}{\text{FPS}_{\text{objetivo}}} = \frac{1}{60} \approx 0.0167 \text{ s}$$

$$t_{\text{restante}} = T_{\text{objetivo}} - \Delta t_n$$

Si el tiempo restante es mayor que 0, se llama a time.sleep con ese valor. Si no, el equipo llega justo a los FPS o no llega a los FPS objetivo y no se espera nada.

1.4 Control del menú con ratón, teclado y gesto de la mano (ControladorMenu)

Descripción: gestiona el menú principal y el de pausa. Admite tres formas de interacción. Con el teclado se usan las teclas W, S, ENTER, ESPACIO y ESC, con

el ratón basta pasar por encima para seleccionar y hacer clic para elegir, y con la mano se mueve un cursor y cerrar el puño sobre un botón equivale a pulsarlo.

Funciones de OpenCV utilizadas y justificación:

- **cv2.EVENT_LBUTTONDOWN:** es la constante que identifica el clic izquierdo dentro del callback del ratón y permite distinguirlo de un simple movimiento.
- **Nota de diseño e inferencia gestual:** no necesita más llamadas a la API. Toma la posición normalizada del detector, la limita al rango de 0 a 1 y la multiplica por las dimensiones virtuales para obtener la posición del cursor. El gesto de pulsar es una transición. Si en el fotograma anterior la mano estaba abierta, ahora está cerrada y el cursor está sobre un botón, se registra la acción de ese botón.
- Los códigos de tecla, que son 13 para ENTER, 32 para ESPACIO y 27 para ESC, vienen de cv2.waitKey y están definidos como constantes en config.py.

1.5 Control de la nave con la mano (ControladorJuego)

Descripción: traduce la posición y el estado de la mano en movimiento y disparos de la nave. Si la cámara ve la mano, manda la mano y, si no, mandan las teclas WASD.

Funciones de OpenCV utilizadas y justificación:

- **Nota de diseño e integración:** este módulo no llama directamente a OpenCV. Usa el resultado del módulo de visión y los códigos de tecla que entrega waitKey:
    - La coordenada horizontal se estira del rango de 0.15 a 0.85 de la cámara al rango de 0 a 1, según la constante MARGEN_MANO. Así se puede llegar a los bordes del campo de juego sin sacar la mano del encuadre. La vertical se usa tal cual, limitada entre 0 y 1.
    - Con la mano cerrada se dispara, respetando un intervalo mínimo de 0.25 s entre disparos.
    - OpenCV solo informa de pulsaciones y no de teclas mantenidas, así que una dirección se considera activa durante 0.1 s después de la última pulsación, según TIEMPO_TECLA_ACTIVA.
    - El tiempo entre fotogramas se limita a 0.1 s para evitar saltos después de una pausa. Los FPS que se muestran se suavizan con un filtro

exponencial que mantiene un 80 % del histórico y aporta un 20 % del valor instantáneo.

1.6 Dibujo del menú (VistaMenu)

Descripción: crea la imagen del menú con el título, el subtítulo, los botones y el cursor de la mano, y escala cada elemento al tamaño real de la ventana sin modificar ningún dato del modelo.

Funciones de OpenCV utilizadas y justificación:

- **cv2.rectangle(canvas, esquina_1, esquina_2, color_boton, -1):** dibuja cada botón como un rectángulo relleno. El botón seleccionado cambia de color con COLOR_BOTON_SELECCIONADO.
- **cv2.getTextSize(texto, cv2.FONT_HERSHEY_SIMPLEX, tamano, grosor):** mide el texto en píxeles para poder centrarlo en el botón o en la pantalla, ya que putText coloca el texto desde su esquina inferior izquierda.
- **cv2.putText(canvas, texto, esquina, cv2.FONT_HERSHEY_SIMPLEX, tamano, color, grosor):** escribe el título, el subtítulo y las opciones. El tamaño y el grosor se multiplican por alto_ventana / ALTO_VIRTUAL para mantener las proporciones cuando se redimensiona la ventana.
- **cv2.circle(canvas, centro, radio, color, -1):** pinta el cursor de la mano como un círculo sólido, rojo si la mano está abierta y naranja si está cerrada, con un radio mínimo de 6 px.
- **Nota de indexación directa con NumPy:** el lienzo se crea con np.full((alto, ancho, 3), color, dtype=np.uint8) y se rellena con el color de fondo del menú.

1.7 Dibujo del juego y composición de sprites con transparencia (VistaJuego)

Descripción: dibuja cada fotograma de la partida con el fondo, las estrellas, las entidades, que son la nave, los enemigos, las balas y las explosiones, y la información en pantalla con la puntuación, el modo de control, el estado de la cámara y los FPS. Las imágenes con canal alfa se mezclan con el fondo para respetar su transparencia.

Funciones de OpenCV utilizadas y justificación:

- **cv2.circle(canvas, centro, radio, (255, 255, 255), -1):** pinta las estrellas del fondo como puntos blancos cuyo radio se escala con la ventana.

- **cv2.imread(ruta, cv2.IMREAD_UNCHANGED):** carga el sprite conservando el canal alfa. Se guarda en un diccionario para no leer el disco en cada fotograma.
- **cv2.resize(imagen, (ancho, alto), interpolation=...):** ajusta el sprite al tamaño de la entidad en pantalla. Se usa INTER_AREA al reducir, que evita el aliasing, e INTER_LINEAR al ampliar.
- **Nota de indexación directa con NumPy:** OpenCV no mezcla transparencias al dibujar, así que se aplica a mano sobre la región del lienzo la mezcla del color del sprite multiplicado por alfa más el color del fondo multiplicado por uno menos alfa. Antes se calcula el recorte visible del sprite para que no falle si la entidad está medio fuera de la pantalla.
- **cv2.rectangle:** dibuja como rectángulo relleno las entidades que no tienen imagen y, con la constante DEBUG activada, el contorno verde de los límites de todas las entidades para comprobar las colisiones.
- **cv2.getTextSize y cv2.putText:** la puntuación se alinea a la derecha midiendo su ancho. El resto de líneas informativas se dibujan en verde en la esquina superior izquierda.

2. Problemas ocurridos y soluciones técnicas

2.1 El espacio de color para detectar la piel

Explicación del problema: para segmentar la mano tenía tres opciones, HSV, BGR o YCrCb. En HSV la piel está en el rojo, que se reparte entre los dos extremos de la escala de tono, el 0 y el 179. Calcular la media del tono obligaba a hacerlo como un ángulo, porque la media normal de 178 y 2 daría 90, que es un color completamente distinto, y además había que partir el rango en dos con inRange. En BGR el color y el brillo van mezclados, y cuando cambia la luz cambian los tres valores a la vez.

Solución implementada: me quedé con YCrCb. Cr y Cb son números normales sin vuelta y el brillo va aparte, así que lo dejo libre en el rango. Basta con la mediana de la zona de la cara y un margen pequeño. El código queda más simple y la detección aguanta mucho mejor las sombras y los cambios de iluminación.

2.2 Paredes y sombras confundidas con piel

Explicación del problema: con un rango de color demasiado amplio, una pared con sombra tenía valores de Cr y Cb parecidos a los de la piel y salía como una gran mancha blanca en la máscara, compitiendo con la mano.

Solución implementada: estreché el margen dinámico alrededor de la mediana de la cara, con Cr a más o menos 1 y Cb a más o menos 10 y el brillo libre. Además lo combiné con un rango fijo de piel mediante cv2.inRange y cv2.bitwise_or, que evita los fallos por sombras sin depender solo de la cara. La selección del bloque descarta las manchas que no siguen a la mano. Tal y como indico en las instrucciones, lo ideal es jugar con un fondo despejado y, a ser posible, blanco.

2.3 La cara y el cuello detectados como mano

Explicación del problema: la cara y el cuello tienen exactamente el color de la mano y, como son más grandes, ganaban siempre cuando se elegía la mancha más grande.

Solución implementada: localizo la cara con un clasificador de Haar, con cv2.CascadeClassifier, y pinto de negro sobre la máscara un rectángulo que la cubre. Lo amplío un 40 % de altura para tapar también el cuello. Mantengo la última cara conocida durante un fotograma, para que un fallo puntual del detector no deje la cara visible en la máscara.

2.4 El brazo desplazaba el centro de la mano

Explicación del problema: la máscara de piel incluía el antebrazo, que puede ser mucho más largo que la mano. El centro de la caja quedaba a medio camino entre los dos y el cursor no coincidía con la posición real de los dedos.

Solución implementada: separar_mano analiza la anchura de la piel fila por fila, localiza la palma, que es la fila más ancha, y corta en la muñeca, que es la primera fila donde la anchura baja al 90 % de la de la palma. Si el brazo nunca se estrecha, se aplica un límite de seguridad de 0.8 veces la anchura máxima.

2.5 Inestabilidad en el reconocimiento de mano abierta y cerrada

Explicación del problema: la solidez por sí sola fallaba con los dedos juntos, porque una mano abierta con los dedos pegados es casi tan compacta como un puño. Además, pequeñas variaciones de la máscara hacían que el estado cambiara entre abierta y cerrada de un fotograma a otro.

Solución implementada: añadí el análisis de los defectos de convexidad con cv2.convexityDefects. Cada valle profundo entre dedos resta 0.08 a la solidez, así que con dos huecos ya se pasa a mano abierta. El valor final se suaviza con un filtro exponencial que mantiene un 80 % del histórico y aporta un 20 % del nuevo. Con eso desaparece el parpadeo del estado y no se nota retardo.

2.6 Saltos entre manchas de piel parecidas

Explicación del problema: cuando aparecían varias manchas con color de piel en la imagen, elegir siempre la mayor hacía que el seguimiento saltara de una a otra de golpe, por ejemplo al acercar la mano a otro objeto.

Solución implementada: doy prioridad a la mancha que ocupa la zona donde estaba la mano en el fotograma anterior, ampliada 50 px por cada lado. Solo si no hay nada ahí recurro a la más grande de la imagen. Un área mínima de 3000 píxeles descarta el ruido.

2.7 Bloqueo de la interfaz y teclas sin estado de pulsación

Explicación del problema: leer la cámara y procesar cada imagen cuesta mucho más que un ciclo de dibujo. Si todo se hacía en el mismo bucle, la ventana se congelaba y los FPS caían. Además, cv2.waitKey solo entrega pulsaciones sueltas y no dice si una tecla sigue pulsada, así que mantener una dirección con WASD producía un movimiento entrecortado.

Solución implementada: separé el programa en tres partes. La cámara tiene un hilo propio que solo guarda el último resultado. Otro hilo actualiza el juego a 60 FPS. El hilo principal se limita a mostrar imágenes y leer teclas, porque la interfaz de OpenCV tiene que usarse desde él. Para el teclado, una dirección se considera activa durante 0.1 s después de la última pulsación recibida.

2.8 Tamaño de ventana variable y texto con tildes

Explicación del problema: al redimensionar la ventana, las posiciones, los tamaños y las colisiones definidos en píxeles dejaban de coincidir. Además, la fuente que incluye OpenCV, cv2.FONT_HERSHEY_SIMPLEX, no dibuja tildes, así que los textos con acentos salían con símbolos raros.

Solución implementada: todo el juego se define en unidades virtuales de 1280 × 720 y solo se convierte a píxeles al dibujar, con escalar_punto, escalar_rect y escalar_longitud. Las coordenadas del ratón hacen el camino inverso. La ventana

corrige además su proporción a 16:9. Para el texto, las constantes de config.py, como los títulos y las opciones de menú, están escritas sin tildes.

2.9 Sprites con transparencia

Explicación del problema: al cargar un PNG con cv2.imread sin más se pierde el canal alfa y la imagen aparece con un fondo negro opaco. Además, las funciones de dibujo de OpenCV no mezclan transparencias por sí solas, y una entidad medio fuera de la pantalla daba errores de dimensiones al copiar la imagen.

Solución implementada: cargo la imagen con cv2.IMREAD_UNCHANGED para conservar el alfa y hago la mezcla a mano con NumPy, multiplicando el color del sprite por alfa y el del fondo por uno menos alfa. Antes calculo la intersección entre el sprite y la pantalla y recorto la parte visible. Las imágenes ya cargadas se guardan en una caché para no leerlas del disco en cada fotograma.

3. Mejoras futuras y líneas de trabajo no implementadas

3.1 Calibración inicial del color de piel sin depender de la cara

Explicación del reto técnico: ahora mismo el rango dinámico de color solo se calcula cuando el detector de Haar encuentra una cara y, hasta que eso pasa, la máscara usa únicamente el rango fijo. La idea sería una pantalla de calibración en la que el jugador ponga la mano dentro de un recuadro y el sistema tome de ella la mediana en YCrCb.

Complejidad prevista: haría falta una fase nueva en la máquina de estados de la aplicación y decidir cuándo hay que recalibrar. También habría que pensar qué pasa si la iluminación cambia durante la partida, porque una calibración manual pierde validez con el tiempo.

3.2 Robustez frente a fondos con tonos parecidos a la piel

Explicación del reto técnico: el sistema es sensible al fondo, y por eso en las instrucciones recomiendo un fondo despejado y, a ser posible, blanco. Una solución sería combinar la máscara de color con una sustracción de fondo, por ejemplo con cv2.createBackgroundSubtractorMOG2, para quedarse solo con lo que se mueve.

Complejidad prevista: un modelo de fondo se contamina cuando la mano se queda quieta, porque acaba pasando a formar parte del fondo. Habría que ajustar su tasa de aprendizaje para no perder la mano en las pausas.

3.3 Reconocimiento de más gestos

Explicación del reto técnico: hoy solo se distingue entre mano abierta y cerrada. Los defectos de convexidad que ya se calculan permitirían contar los dedos extendidos y asignar acciones nuevas, como pausar el juego con un gesto o cambiar el tipo de disparo.

Complejidad prevista: contar dedos con fiabilidad es delicado, porque depende de cómo esté girada la mano y de si los dedos están juntos. Habría que filtrar los defectos por ángulo además de por profundidad y añadir un tiempo de confirmación para evitar acciones sin querer.